

Ch8. 성능 최적화

8.1 성능 최적화

1. 데이터를 사용한 성능 최적화: 데이터 갯수 늘림

- 최대한 많은 데이터 수집: 딥러닝 알고리즘은 머신 러닝 알고리즘에 비해 데이터 갯수의 증가에 비해 성능 증가율이 큼

- 데이터 생성하기: 많은 데이터를 수집 할 수 없으면 데이터를 만들어서 사용

- 데이터 범위 조정하기: 활성화 함수로 시그모이드 사용-> 데이터셋 범위를 0~1의 값을 갖도록 조정할 수 있음, 활성화 함수로 하이퍼볼릭 탄젠트 사용-> 데이터셋 범위를 -1~1의 값을 갖도록 조정할 수 있음

- 정규화, 표준화 등...

2. 알고리즘을 이용한 성능 최적화: 유사한 용도의 알고리즘들을 선택해서 모델 훈련 -> 최적의 성능을 보이는 알고리즘 선택

머신 러닝: SVM, K-nearest neighbor 등..

시계열 데이터: RNN, LSTM, GRU 등..

3. 알고리즘 튜닝을 위한 성능 최적화: 성능 최적화에서 가장 많은 시간이 소용되는 부분.

모델 선택 -> 하이퍼파라미터 변경 및 훈련 -> 최적의 성능 도출

선택할 수 있는 하이퍼파라미터:

- 진단: 성능 향상이 어느 순간 멈추면 원인 분석 필요. 모델에 대한 평가를 사용해서 문제 진단 가능(ex. 과적합, 과소적합 등..)

-가중치: 가중치에 대한 초기값은 작은 난수 사용. 오토인코더 같은 비지도 학습을 이용해 사전 훈련(가중치 정보를 얻기 위한 사전 훈련)을 진행한 후 지도 학습을 진행하는것도 방법

-학습률: 모델의 네트워크 구성에 따라 다름. 초기에 매우 크거나 작은 임의의 난수를 선택하여 학습 결과를 보고 변경해나가야 함. 네트워크 계층이 많이 없으면 학습률 작게 설정해야함.

-활성화 함수: 활성화 함수의 변경 시 손실 함수도 변경해야 하는 경우가 많으므로 신중히 변경. 다루고자 하는 데이터 유형 및 데이터로 어떤 결과를 얻고 싶은지 이해한 후 변경 필요.

-배치와 에포크: 최근 딥러닝 트렌드는 큰 에포크와 작은 배치. 최적화를 위해 다양한 테스트

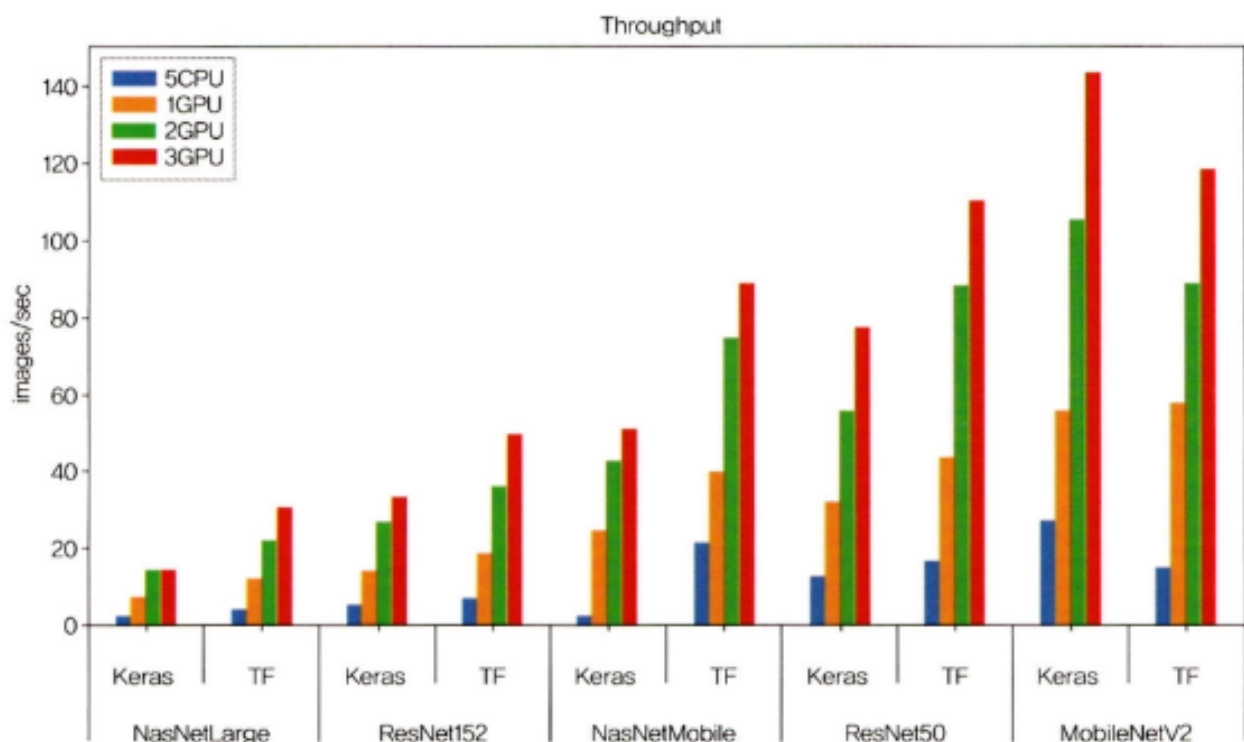
-옵티마이저 및 손실 함수: 일반적으로 옵티마이저는 확률적 경사 하강법 많이 사용. Adam, RMSProp도 괜찮음. 다양하게 적용한 후 최고 성능 선택

-네트워크 구성(Topology): 네트워크 구성을 변경해 가면서 성능 테스트(ex. 넓은 네트워크 / 깊은 네트워크 등..)

4. 앙상블을 이용한 성능 최적화: 앙상블 - 모델 2개 이상 섞어서 사용.

8.2 하드웨어를 이용한 성능 최적화

cpu&gpu차이

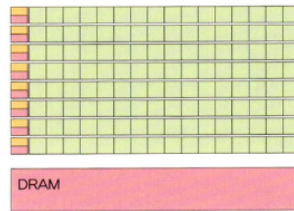
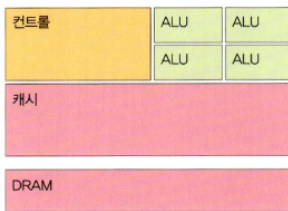


CPU: 명령어가 입력되는 순서대로 직렬처리 -> ALU갯수가 많이 필요 없음

GPU: 병렬처리 -> 연산 담당하는 ALU여러개 필요

CPU

GPU



CUDA: NVIDIA에서 개발한 GPU개발 툴.
많은 양의 연산 동시에 처리 가능

▼ 표 8-1 GPU 모델 정보

구분	GEFORCE GTX 1080	GEFORCE GTX 1070	GEFORCE GTX 1060(3GB)	GEFORCE GTX 1050
Core	2560	1920	11512	640
Clock(MHz)	1607	1506	1506	1354
Memory	11GB	8GB	6GB	4GB
Memory Speed	10Gbps	8Gbps	8Gbps	7Gbps

8.3하이퍼파라미터를 이용한 성능 최적화

1. 배치 정규화

- 정규화(normalization): 데이터 범위를 사용자가 원하는 범위로 제한, feature scaling이라고도 함

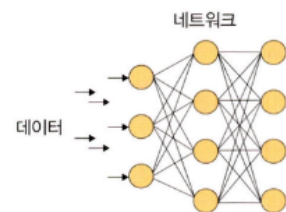
- 규제화(regularization): 모델 복잡도를 줄이기

위해 제약을 두는 방법. (데이터가 네트워크에 들어가기 전에 필터 적용)

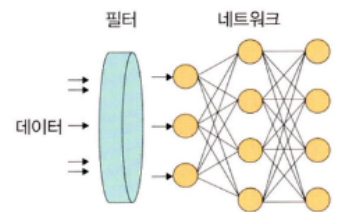
-드롭아웃

-조기 종료

-표준화(standardization): 기존 데이터 평균=0, 표준편차=1인 데이터로 만드는 방법(aka. Standard scaler, z-score normalisation)



필터가 적용되지 않은 경우



필터가 적용된 경우

-배치 정규화: 데이터 분포가 안정-> 학습 속도 증가. 기울기 소멸, 기울기 폭발 같은 문제 해결

1. 미니 배치 평균 계산 -> 2. 미니 배치 분산&표준편차 계산 -> 정규화 수행 -> 4. 데이터 분포 조정

단점: 배치 크기가 작을 때는 정규화 값이 기존 값과 다른 방향으로 훈련될 수 있음

RNN은 네트워크 계층별로 미니 정규화를 적용해야 하기 때문에 오히려 모델 복잡도 증가

2. 드롭아웃: 훈련할 때 일정 비율의 뉴런만 사용, 나머지 뉴런에 해당하는 가중치는 업데이트 하지 않는 방법. 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습.

과하게 훈련 데이터셋 학습할 경우 -> 테스트 데이터셋에서 오류 증가(과적합)

훈련 시간이 길어지는 단점. 모델 성능 향상 장점

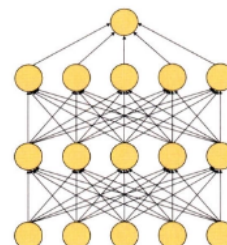
3. 조기 종료를 위한 성능 최적화

조기종료: 뉴럴 네트워크가 과적합을 피하는 규제 기법. 에포크마다 검증 데이터에 대한 오차를 측정해서 모델의 종료시점 제어.

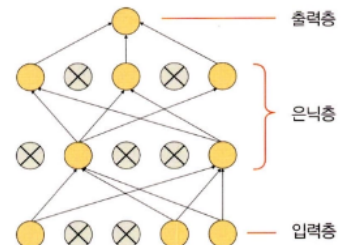
과적합 발생 전: 학습에 대한 오차, 검증에 대한 오차 모두 감소.

과적합 발생 후: 훈련 데이터셋에 대한 오차 감소, 검증 데이터셋에 대한 오차 증가.

=> 조기 종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습 멈추도록 조정



일반적인 신경망



드롭아웃이 적용된 신경망